Session 5: Subagents, Hooks, and Final Project

Spawning focused agents, event-driven automation, and an end-to-end research artifact

Brady Allardice

UPF & IBEI

Start-of-Session Ritual: Session Folder

First, pull the course repo and copy today's working folder.

Replace [my_workspace] with wherever you keep your local course files.

```
# 1. Pull the latest course repo.
cd ~/claude-code-workshop && git pull

# 2. Copy today's session folder into your workspace.
cp -r session_4/ [my_workspace]/session_4/
```

Don't have the course repo yet?

`git clone https://github.com/bradyallardice/claude-code-workshop.git`

If `git pull` fights back, use the Session 4 recovery: `git stash -u`; `git branch backup-s5`; `git fetch origin`; `git reset --hard origin/student`; `git stash pop`.

Start-of-Session Ritual: Slides

Now copy the slides. Choose exactly one option.

Option A: you already have `[my_workspace]/slides/`

```
cp slides/session5_slides.pdf [my_workspace]/slides/
```

Option B: you do not have a `slides/` folder yet

```
cp -r slides/ [my_workspace]/slides/
```

Do not run both. Then open today's workspace:

```
cd [my_workspace]/session_4 && code .
```

Today: Agents, Hooks, and a Paper

Three parts.

1. **Agents.** Subagents, named specialists, and multi-agent orchestration — when to spawn one, how to define your own, how they compose.
2. **Hooks.** Shell (terminal) commands that run automatically on events (file writes, tool use, session start) — enforcement that doesn't depend on Claude remembering.
3. **Putting it together.** Start from an unfamiliar paper, use a Zotero library to write the literature review, then build a short replication + extension note.
4. **And Beyond.** A few notes additional Claude Code capabilities.

Subagents

A focused agent spawned for a bounded job —

the right tool for the parallelizable, scoped work that already shows up in your pipeline.

Concrete patterns, then named teams.

What a Subagent Is

Claude Code can spawn another Claude Code instance inside your current session — a subagent.

- It has its own context window
- It gets a scoped task and a scoped tool set
- It returns a summary to your main conversation — not its raw trace

What that buys you:

1. **Parallelism** — run three searches at once instead of one after the other
2. **Context hygiene** — a big messy exploration stays out of your main context budget

Think of it as

Delegating a bounded sub-task to a focused colleague who returns a clean, structured answer — so your main conversation stays on the high-level work.

Where Subagents Shine

Reach for a subagent when the subtask is:

- **Parallelizable** — independent work that doesn't need to be sequenced
- **Bounded** — clear inputs, clear outputs, no back-and-forth
- **Independently verifiable** — you can check the result without reading the full trace

Concrete research use cases:

- **Multi-database literature searches** — one agent per database, all running in parallel
- **Coding many documents** against a fixed rubric
- **Parallel robustness checks** across independent specifications
- **Running the same sanity-check** script over many datasets
- **Exploring a codebase** — send a subagent to map out the files, keep the summary in your main context

How to Trigger a Subagent

You don't have to set anything up — just ask.

Claude Code ships with a general-purpose subagent built in. To spawn one, describe the work in plain English:

- *"Send a subagent to map the directory structure"*
- *"Use a subagent to find every place we set the random seed"*

For parallelism, say so explicitly.

- *"Spawn three agents* **in parallel** *to search Zotero, Google Scholar, and Connected Papers"*
- "In parallel" / "at the same time" / "concurrently" all signal that the agents should run in one batch rather than sequentially

No setup required

The general-purpose agent ships with Claude Code. Defining your own named agent (later this session) is for when you want to reuse a scoped role across many sessions.

Using a Generic Subagent in Practice

Worked example: one subagent, one bounded local task.

Your prompt to Claude:

```
Spawn a subagent to read the slide PDFs in slides/
for sessions 1-4 and return a deduplicated list of every unique terminal
command I'll need across the course.

Output as: command | what it does | first session
it appears in.
```

What Claude does:

- Spawns one general-purpose subagent and hands it the four slide PDFs in `slides/`
- The subagent does the reading, extraction, and dedup in its own context
- You get back the table; the slide text never lands in your main conversation

Using Subagents Well

Three habits that make subagents pay off:

1. **Brief them like a contractor.** Concrete deliverable, all the context they need up front — they don't see your conversation history.
2. **Ask for structured output** (bullet list, table, JSON). You'll be consuming the result; make it easy to read and verify.
3. **Verify independently.** You get a summary, not the full trace. Spot-check the way you would a student's work.

The payoff

A literature sweep, robustness battery, or repo survey that would have taken an hour finishes in minutes — and doesn't eat into your main conversation's context. That's what makes them worth reaching for.

What a Predefined Subagent Looks Like

A small markdown file — author it once, Claude spawns it many times.

Where it lives: `~/.claude/agents/<name>.md` (user-level) or `.claude/agents/<name>.md` (project-level).

What you specify (metadata + body):

- **Identity** — `name` and a one-line `description`. The description is what Claude reads to decide when to spawn this agent.
- **Tool allowlist** — exactly which tools it can use (e.g. `Read` only, no `Bash`).
- **Model** — Opus, Sonnet, or Haiku.
- **System prompt** — the body of the file: role, disposition, expected output format.

Plain text — metadata sits between two `---` lines at the top of the file; the system prompt is everything below.

Building a Specialized Agent

Worked example: turn "audit my regression script" into a reusable role.

Walking through the four fields:

1. **Name + description.** What it does, plus the one-line trigger Claude reads to decide when to spawn it.
   "Audit a regression script for silent issues."
2. **Tool allowlist.** `Read`, `Grep`, `Glob` — read-only. A critic that can fix code is no longer a critic.
3. **Model.** `haiku` — fast and cheap, fine for systematic pattern-matching.
4. **System prompt.** Disposition + output contract: *"You're a skeptical reviewer. Flag invalidating issues. Return: line, issue, why. Don't propose fixes."*

Save as `~/.claude/agents/spec-critic.md`. (The capstone template, later today, ships its own project-level `.claude/agents/spec-critic.md`: the same idea, already written.)

Now in any session:

- Claude auto-spawns it when a task matches the `description`
- Or invoke explicitly: *"use `spec-critic` on `<your regression script>`"*, e.g. the estimation script you wrote in Session 2
- Role is enforced — if it suggests a fix, it broke its own contract

Why Define a Named Subagent at All?

If skills are reusable, why not just spawn
ad-hoc subagents that pick up the right skill?

Why Define a Named Subagent at All?

The framing

Skills are knowledge. Subagents are roles.
A role bundles a disposition, a tool scope, and an invocation contract — and then draws on skills to execute.

Three things a role gives you that a skill can't:

1. **Tool scope — enforced.** A code-review subagent has read-only access, no bash. Skills are advisory; tool scopes aren't.
2. **Identity, not advice.** The system prompt frames the whole disposition — "find problems, don't fix them." Subagents *are* the disposition.
3. **A stable invocation contract.** Delegate code review → get a structured critique. Ad-hoc means re-specifying the contract each time.

Pre-define a subagent when the **role** is the reusable unit, not just the know-how.

Multi-Agent Workflows

When one specialist isn't enough.

Compose subagents into pipelines, fan-outs, and critic-builder loops.

Three patterns, one worked example.

Three Composition Patterns

Most multi-agent workflows are one of three shapes.

1. **Fan-out / map-reduce.** N inputs, one specialist per input, merge at the end.
   Example: code 50 interview transcripts against the same rubric — one agent per transcript, then synthesize.

2. **Pipeline.** Each stage's output is the next stage's input. Named and generic agents can mix.
   Example: `spec-critic` flags issues → generic agent drafts fixes → `spec-critic` re-audits.

3. **Critic-builder loop.** One agent produces, another reviews, the producer revises.
   Example: a paragraph drafter ↔ a paragraph-reviewer, two or three rounds.

A Pipeline in Practice

Worked example: hardening a regression script in three stages.

Your prompt to Claude:

```
Run this pipeline on <your regression script>:
  1. Use spec-critic to list every silent issue.
  2. Spawn a subagent to draft minimal-diff fixes
     for each issue.
  3. Re-run spec-critic on the patched script.
Return: original issues | proposed fixes | residual issues.
```

What makes this work:

- Each stage has a stable input/output contract — so they compose
- Two `spec-critic` calls share a role but run in fresh contexts — consistent without state
- Your main conversation only sees the final summary; intermediate traces stay isolated

One Agent or Many?

Default to one agent. A team has to earn its keep.

Stick with one agent when:

- The task is bounded enough to fit one context window
- One disposition is enough — no producer/critic split
- You're still exploring — you don't yet know the right stages
- Orchestration overhead would dominate the actual work

Reach for a team when:

- The work has naturally distinct roles (writer + reviewer; auditor + fixer)
- Independent verification is the point — a separate set of eyes
- You'll run the workflow many times — the setup cost amortizes
- N independent inputs need the same treatment in parallel

The test

Each extra agent is a stage that can drift. Add one only if it does something the previous one demonstrably can't.

Hooks

Shell (terminal) commands the harness runs automatically on events — file writes, tool calls, session start. Enforcement that doesn't depend on Claude remembering.

Concept, one example, when to reach for one.

What a Hook Is

A hook is a shell command wired to a Claude Code event. Configured in `.claude/settings.json`; run by the harness, not by Claude.

Common events:

- **PreToolUse** — before a tool runs (can block it)
- **PostToolUse** — after a tool runs
- **SessionStart** — at the start of a session
- **UserPromptSubmit** — when you submit a prompt
- **Stop** — when Claude finishes responding

What this buys you:

- **Enforcement, not advice.** The hook runs whether Claude remembers it or not.
- **Auditability.** Log every tool use, every edit, every Bash call.
- **Automation.** Format on save, refresh derived files, notify on completion.

Worked Example: A Research Audit Log

Goal: every Bash command Claude runs gets logged with a timestamp — automatically, every session.

Add to `.claude/settings.json`**:**

```
{ "hooks": { "PreToolUse": [ { "matcher": "Bash",
    "hooks": [ { "type": "command",
      "command": "echo \"$(date '+%F %T') $(jq -r '.tool_input.command')\" >> ~/research_log.txt" } ] } ] } }
```

What happens: Before every `Bash` call, the harness hands the hook a JSON description of the call, the hook pulls out the command with `jq`, and appends a timestamped line to your research log. Claude can't bypass it — the harness, not the model, runs hooks.

Needs `jq` installed. It records the *command*, not what the command printed or changed.

Installing a Pre-Built Hook: `claude-ignore`

Many useful hooks already exist — you don't have to write your own.

What `claude-ignore` does:

- A `PreToolUse` hook on `Read` that blocks files matched by a `.claudeignore` (gitignore-syntax patterns; comments and blank lines are ignored)
- Walks up from the current directory to root, merging every `.claudeignore` it finds — different rules at different folder levels
- On a match, exits with code 2: hard block. Claude sees the failure and can't bypass it
- Use for `.env`, `*.secret`, raw data — anything you don't want in the model's context

Install + initialize:

```
npm install -g claudeignore
cd [my_workspace]/my_project && claude-ignore init
```

`init` creates a `.claudeignore` and wires the hook into `.claude/settings.json`.

Source: `github.com/li-zhixin/claude-ignore`. Requires Node.js: if `npm` is not found, install Node first. Try it in a throwaway project folder.

Hook, Skill, or Agent?

Three layers of automation, three different things.

The framing

Skills are knowledge. Subagents are roles. Hooks are guarantees.

Reach for a:

- **Skill** when Claude needs to know *how* to do X.
- **Subagent** when you want a delegated worker with a scoped role.
- **Hook** when something must happen regardless of what Claude remembers.

Hooks are the only one of the three the harness enforces. Skills are advice; subagents have agency; hooks are mechanical.

Capstone

Create a full research note
in a team of two.

Work in a local git repository: branches, merges, and review by diff. No GitHub account needed.

Steps: read the assigned paper,
write a short literature review,
replicate one published figure,
run one small extension.

Assigned paper: Magistro, Borwein, Alvarez, Bonikowski, and Loewen (2026),
"Attitudes Toward Artificial Intelligence (AI) and Globalization:
Common Microfoundations and Political Implications," *AJPS*.

Use Claude Code as a research assistant, not as a vending machine: learn the paper, direct the task, inspect the code, verify the output, and interpret the result.

Capstone Flow

- **Step 1: Orient yourself.** Read the article, notes, and codebook.
- **Step 2: Team repo.** Pair up, copy the template into a local git repo, use branches.
- **Step 3: Zotero.** Import the library; keep Claude out of local BibTeX.
- **Step 4: Literature review.** Search Zotero, draft, verify claims.
- **Step 5: Validate data.** Check structure before analysis.
- **Step 6: Replicate Figure 1.** Prompt, plan, code, inspect, commit.
- **Step 7: Extension + note.** Run one extension and finish the paper.
- **Step 8: Reflect.** Fill in `AI_WORKFLOW_REFLECTION.md`.
- **Step 9: Transfer.** Decide what becomes a prompt, skill, agent, or hook.

The deliverable is not just the PDF. It is the PDF plus evidence that you stayed in control of the workflow.

Step 0: What Is Provided

Course infrastructure

- A template repo with data, starter scripts, paper skeleton, and README
- The paper, and a Zotero library (`zotero/capstone_library/capstone_library.rdf`) with the synthesis notes and PDFs attached
- Hooks that protect `reference_code/`, `main`, and ignored files
- General skills: data validation, robustness, tables, references, prose review
- Reviewer subagents: code critic, results trace, PR referee, note reviewer

Your work

- Read the paper and the synthesis notes
- Write the literature review from the Zotero library
- Write the replication instruction yourself
- Decide what Claude may infer and what you must specify
- Read the code and check the output
- Interpret the extension
- Turn it into a 2–3 page research note in your repo
- Explain which AI workflow pieces would transfer to your own work

Step 1: Orient Yourself with the Paper

Start with the assigned paper and the synthesis notes.

Assigned paper PDF: `zotero/pdfs/magistro_et_al_2026_ai_globalization.pdf`
Other PDFs in `zotero/pdfs/`**:** supporting readings for the literature review.
Data PDF: `data/Codebook_AJPS.pdf` is the variable codebook.

Use the paper, the Zotero library, and the synthesis notes attached to its papers (you import them in Step 3). Your first task is to map:

- What is the paper's research question?
- What is the paper's core contribution?
- What treatment, attributes, and outcome are being analyzed?
- What does Figure 1 estimate, and why is it central?
- What does the assigned extension change or add?
- Which literatures does the paper claim to speak to?

Goal: know what the figure and extension are supposed to teach before you ask Claude to write code.

Step 2: Make a Local Git Repository

Copy the template into a new folder and put it under git. No GitHub account is needed.

In a terminal, from `[my_workspace]/session_4` (where the ritual left you). Run these yourself, not through Claude Code: the main-branch hook blocks Claude from committing the starter files.

```
cp -r capstone-template [my_workspace]/capstone
cd [my_workspace]/capstone
git init -b main
git add .
git commit -m "Add capstone starter files"
```

Check: `ls -a` shows `README.md`, `CLAUDE.md`, `.claude/`, `starter/`, `paper/`, `data/`, and `zotero/`. `git log --oneline` shows one commit.

If `git init -b main` is rejected (git older than 2.28): `git init`, then `git symbolic-ref HEAD refs/heads/main`, before the first commit. Open the folder with `code .` and run everything from here.

`session_4/capstone-template/` is the source; your copy is the workspace. Do not edit the template.

Step 2b: Branch and Review

Working alone: one branch per lane. Working as a pair: each partner takes a lane and reviews the other's.

```
git switch -c lit/<name> # paper + literature lane
git switch -c analysis/<name> # data + analysis lane
```

Fill in `ROLES.md` **first.** Then review before merging, from the branch you are merging:

```
git diff main..HEAD # read every changed line
> Use the pr-referee subagent on this branch's diff
git switch main && git merge lit/<name>
```

For a pair on one machine, take turns; for two machines, exchange the repo folder or use any shared remote you already have (the README has an optional GitHub variant). **Always** read the diff before the merge. Commit to a branch, never directly to `main`.

Step 3: Zotero

The literature review uses a prebuilt Zotero library.

Import `zotero/capstone_library/capstone_library.rdf`. It carries the 11 papers, their PDFs, and the synthesis notes attached to each paper.

The workflow:

1. Zotero: **File** → **Import...** → **A file**, choose the `.rdf`. Tick "Place imported collections and items into new collection" and "Copy files to the Zotero storage folder".
2. Keep the `files/` folder *next to* the `.rdf` while importing, or the PDFs do not come in.
3. Confirm the notes and PDFs imported and the MCP can find the papers.
4. Ask Claude to search Zotero, not to read local library files.

Why

A local `.bib` file lets Claude pattern-match citations without understanding the literature. Zotero forces the workflow closer to the thing you would do in a real project: search, inspect, cite, and verify.

`zotero/capstone_library.bib` is a citation-only copy: no notes, no PDFs. Do not import it as well.

Step 4: Write the Literature Review

The synthesis notes are already in your Zotero library. Use them.

A useful first prompt:

> *Read the synthesis notes attached to each paper in the Zotero library. Across all notes, identify the three or four recurring mechanisms that connect automation and trade shocks to political preferences. Then draft 2–3 paragraphs that position the capstone paper relative to those mechanisms. Every sentence must be traceable to a specific note.*

Then:

- Check that every citation supports the sentence attached to it.
- Add any connections the model missed that you saw in the highlights.
- Cut anything that doesn't directly motivate the capstone's design.

Step 5: Validate the Data

Before replication or extension code, audit the analysis dataset.

Run this from the capstone folder (the repo root):

```
use spec-validator on data/clean_AJPS.csv
```

Then decide what each warning means:

- Are treatment, task, and respondent counts plausible?
- Are missing values expected or analysis-changing?
- Is the outcome scale coded the way the paper describes?
- Does the data support the reshape needed for Figure 1?

Validation is not asking Claude if the data is fine. It is deciding whether the data structure matches the design.

Step 6: Replicate Figure 1

Before Claude writes code, write the instruction yourself.

It should specify:

- **Input:** `data/clean_AJPS.csv`
- **Reshape:** one respondent-task per row
- **Estimand:** marginal means by AI vs. Offshoring treatment
- **Visual target:** four stacked panels matching Figure 1
- **Output:** `paper/figures/figure1_replication.png`
- **Checks:** row counts, task counts, treatment counts before save

```
Ask Claude for a plan first. If the plan chooses a join,
changes the sample, or skips verification, reject it.
```

Step 7: Run the Extension and Finish the Note

Now run the assigned extension and turn the work into a research note.

Use one starter file, depending on your language:

```
starter/extend_TEMPLATE.R
starter/extend_TEMPLATE.py
starter/extend_TEMPLATE.do
```

Then:

- Fill in the assigned extension from `CLAUDE.md`.
- Save figures to `paper/figures/` and tables to `paper/tables/`.
- Edit `paper/paper.tex`: intro, design, results, discussion.
- Use reviewer agents before merging: `note-reviewer`, `results-trace-auditor`, and `pr-referee`.

Step 8: Reflect on the Workflow

The capstone uses course-provided infrastructure. Your own research will not.

In `AI_WORKFLOW_REFLECTION.md`, answer:

- What did you delegate to Claude?
- What did you personally verify?
- What did you refuse to let Claude decide?
- What would belong in `CLAUDE.md` for your own paper?
- What repeated convention would become a skill?
- What reviewer role would become a subagent?
- What safety rule would become a hook?

What I am looking for

Not "I used all the tools." I want to see that you can choose the right layer for the next project.

Step 9: Why Figure 1 Is Not a Skill

Inside this capstone, reproducing Figure 1 happens once.

It is project-specific: one paper, one dataset, one reference figure, one visual target.

So it should not be an installed skill. The transferable move is not "run my Figure 1 skill." The transferable move is:

1. Read the reference code
2. State the reshape target and estimand
3. Specify what the output should look like
4. Tell Claude what checks to print before saving
5. Review the plan, the code, and the final figure

The routing lesson

Some work is one-off for this project but still teaches a reusable pattern for your own research.

Beyond the Terminal

Four power-user features worth knowing about.

1. **Claude Code in the cloud.** Run Claude on Anthropic's machines, not yours.
2. **Teleport and remote control.** Move a session between laptop and phone — two slides, because it is the most useful and the least obvious.
3. **Scheduled and recurring tasks.** `/loop` and `/schedule`.
4. **Customizing your environment.** Status line, colors, keybindings, voice, output styles.

None of these are required for research work. They are optional comforts that compound once you use Claude every day.

Source: support.claude.com — Claude Code power-user tips.

Claude Code in the Cloud

What it is. The same Claude Code, but running on an Anthropic-hosted sandbox instead of your laptop. You launch it from the Claude app or `claude.ai/code`, point it at a GitHub repo you've connected, and it works there.

Why it is useful.

- Long jobs (replications, sweeps, paper rebuilds) keep running with the lid closed.
- Several sessions in parallel without burning your local CPU or stepping on your worktrees.
- It is how you dispatch work from a phone or browser when you don't have a terminal.
- **Cowork** is the variant that can also reach into your Desktop app to act on local context (files, Slack, calendar) when you authorize it.

```
# At claude.ai/code, open the "Code" tab, pick the repo, and prompt:
"On a new branch, rerun the full extension grid in session_4/
 and open a PR with the updated tables and figures."
```

Trade-off. Cloud sessions are great for repo-shaped work. Anything that depends on your local files, your local data, or unpushed commits stays on your machine.

Teleport and Remote Control: What They Are

Two ways to keep one session continuous across devices.

- **Teleport.** A handoff. Your current Claude Code session — conversation, plan, todos, working directory — moves to another device (terminal, desktop app, web, or phone). The original session ends; the new one picks up exactly where it left off.
- **Remote control.** A pairing. The session keeps running where it started (usually your laptop), and your phone becomes a remote that can read the output and send prompts. The work still happens on the laptop.

Why it is useful for a researcher.

- You start a long agent run at your desk and want to monitor it on the train without restarting context.
- You think of a follow-up prompt while away from the laptop and want it queued, not forgotten.
- You don't want three half-finished sessions across three devices — just one canonical thread.

Teleport and Remote Control: How to Use Them

Two short flows.

Flow 1 — Teleport (hand the session off).

```
# At your desk, mid-session:
/teleport
# Pick the destination device (phone / web / desktop app)
# from the prompt. The session continues there;
# this terminal session ends.
```

Flow 2 — Remote control (drive from your phone).

```
# Laptop, before walking away:
/remote-control
# Pair your phone (one-time, via the Claude mobile app).
# Now the laptop keeps running the agent;
# the phone shows output and can send prompts.
```

Rule of thumb. Teleport when you are leaving the laptop behind. Remote-control when the laptop is the right machine for the work but you want the phone as a window.

Scheduled and Recurring Tasks

What it is. Two ways to make Claude run on a clock instead of on your keystroke.

- `/loop` — repeat a prompt or slash command locally, on an interval, for up to a few days.
- `/schedule` — create a cloud-side cron job that runs even when your machine is off.

Why it is useful for research.

- Re-run a robustness battery every time results change.
- Sweep your Zotero feed for new papers and append a digest to a notes file.
- Babysit PRs and CI without sitting in front of the terminal.

```
# Every 30 minutes, recheck open PRs and address review comments
/loop 30m /babysit-prs

# Cloud cron: every Monday 9am, summarize new Zotero items into NOTES.md
/schedule
```

Use sparingly. A loop that runs an unverified workflow is a loop that produces unverified output on a schedule.

Customizing Your Environment

What it is. Small quality-of-life knobs in the CLI and Desktop app.

- `/config` — theme, model, output style (e.g. "Explanatory" or "Learning").
- `/statusline` — show model, working directory, context usage at a glance.
- `/color`, `/keybindings` — distinguish parallel sessions, rebind keys.
- `/terminal-setup` — enable `Shift+Enter` for newlines.
- `/voice` — dictate prompts instead of typing.

Why it is useful. Friction compounds. If two sessions look identical and you paste the wrong thing into the wrong worktree, the cost is real.

```
# Two parallel worktrees, visually distinct
# Terminal A:
/color blue
# Terminal B:
/color red

# Switch to a verbose, teaching-style mode while learning a new tool
/config # then pick output style "Explanatory"
```

Pick One, Not All

Don't try to adopt all of these in the same week.

A reasonable order for a researcher:

1. **Status line and colors** first — cheap, prevents real mistakes across worktrees.
2. **Cloud Claude Code** when you have a job that genuinely doesn't fit on your laptop, or when you want to dispatch work from a phone.
3. **Teleport / remote control** the first time you find yourself wishing you could keep the agent working while you're away.
4. `/loop` **and** `/schedule` only once a workflow is verified — never to discover whether it works.

The best power-user setup is the smallest one that removes a friction you actually feel.

Web Scraping with Claude Code

Claude Code can write and run scraping scripts in your session.

A typical workflow:

1. Describe the target page and what you want to extract.
2. Claude writes a script (Python with `requests` + `BeautifulSoup`, or `playwright` for dynamic pages).
3. You inspect the output before saving anything.
4. Claude adjusts selectors if the structure doesn't match.

What it handles well:

- Static HTML pages — legislative records, committee lists, government databases
- Paginated tables where the pattern repeats
- Light dynamic content with `playwright` or `selenium`

Always check the site's `robots.txt` and terms of service before scraping. Pause between requests. Don't hammer public servers.

Scraping: A Worked Prompt

Example: scrape a table of legislative votes.

```
The page at [URL] has a table of roll-call votes. Each row has:
  legislator name, party, constituency, and vote (Y/N/Abstain).
The table spans multiple pages (next-page button at bottom).

Write a Python script that:
  1. Fetches each page with a 1-second pause between requests
  2. Parses the table using BeautifulSoup
  3. Saves the combined result to data/votes_raw.csv
  4. Prints row count and NA counts before saving

Do not run it yet - show me the plan first.
```

Then: review the plan, approve, run, inspect the CSV, and commit. Same workflow as replication.

The Claude API

When Claude Code is not the right tool.

Claude Code is an interactive assistant. The **Claude API** (via the Anthropic Python or TypeScript SDK) is for when you need to:

- Run the same prompt over **thousands of observations** — coding open-ended survey responses, classifying documents at scale
- Embed Claude in a **pipeline** — a script that processes each row and writes results to a dataframe
- Control **exact model, temperature, and token budget** for reproducibility
- Build a **custom tool** that wraps Claude for a specific recurring task

The distinction

Claude Code is for you-and-Claude working together. The API is for Claude working at scale, without you in the loop for each item.

Claude API: A Minimal Research Example

Coding 500 open-ended survey responses.

```
import anthropic, pandas as pd

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from env
df = pd.read_csv("data/survey_responses.csv")

def code_response(text):
    msg = client.messages.create(
        model="claude-opus-4-7",
        max_tokens=64,
        messages=[{"role": "user", "content":
            f"Classify this response as Positive, Negative, or Neutral.\n"
            f"Response: {text}\nLabel:"}]
    )
    return msg.content[0].text.strip()

df["label"] = df["open_ended"].apply(code_response)
df.to_csv("data/survey_coded.csv", index=False)
```

In practice: add prompt caching (`cache_control`) for the system prompt, validate a random sample of labels, and log the model version and date for reproducibility.

API vs. Claude Code: When to Use Which

Use Claude Code when:

- You are exploring, iterating, or debugging
- The task requires your judgment at each step
- You want to write a scraper, plan an analysis, or review output interactively
- One-off or low-volume tasks

Use the API when:

- You have a fixed prompt and many inputs
- You need the results in a dataframe or database
- Reproducibility requires logging exact model and parameters
- You are building a tool others will run without Claude Code installed

The API is not more powerful than Claude Code — it is differently suited. Most research workflows use both: Claude Code to design the prompt and test it, the API to run it at scale.